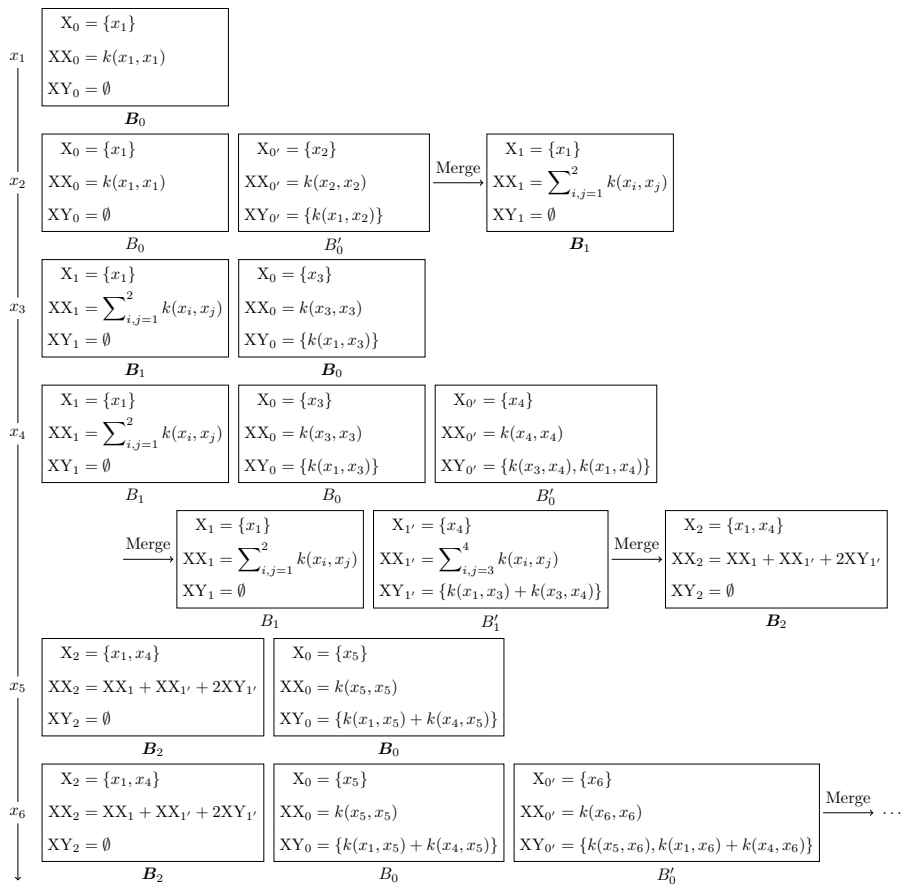

Figure 2: Set up of data structure with subsampling upon inserting $x_1, \ldots, x_6$. MMDEW stores the windows in bold face at the end of the merge operations. Observations x_2 and x_3 are not stored explicitly due to the sampling applied. x_4 is split into two lines for readability. See Example 2 for a detailed discussion.

The algorithm now recursively merges the windows, that is, B_0 and $B_{0'}$ become $B_{1'}$, and B_1 and $B_{1'}$ then become B_2. Upon receiving x_5, the algorithm creates a new window B_0, storing x_5, the kernel evaluation $k(x_5, x_5)$, and the interaction of x_5 with $\{x_1, x_4\}$ from B_3. We conclude the example with x_6, which leads to the creation of a new window B'_0. As in steps 2 and 4, B_0 and B'_0 will now be merged to obtain B_1.

Algorithm 1 has a runtime cost of $\mathcal{O}\left(\log^2 t\right)$ per insert operation and a total memory complexity of $\mathcal{O}\left(\log t\right)$. This allows it to scale to very large data streams. Nevertheless, if one strictly requires constant time and memory, one can simply limit the number of windows at the expense of detecting changes only up to a certain time in the past. In the latter configuration, MMDEW fulfills the requirements for streaming algorithms laid out by Domingos & Hulten (2003).

5 Experiments

This section showcases our approach on synthetic data (Section 5.1) and on streams derived from real-world classification tasks (Section 5.2). We ran all experiments on a server running Ubuntu 20.04 with 124GB RAM, and 32 cores with 2GHz each.

5.1 Synthetic data

To evaluate the average run length (ARL) and the mean time to detection (MTD) in a controlled environment, we first conduct experiments on synthetic data, comparing MMDEW to the MMD estimate (1) as baseline.[6] We also compare the runtime of MMDEW to that of existing change detectors. For an in-depth comparison of the ARL and MTD trade-off for kernel-based approaches with optimally chosen thresholds, see Appendix D.1. For a comparison of MMDEW with univariate approaches, we refer to Appendix D.2.

ARL and MTD. The ARL quantifies the expected number of observations processed before a change detector flags a change, assuming H_0 holds. In the static setting, this corresponds to the type I error. Formally, for $\tilde{T}$ corresponding to the stopping time captured by Algorithm 1, that is, (9) with subsampling applied, we are interested in $\mathbb{E}_{H_0}\tilde{T}$.

The error under the alternative (H_1 holds) is captured by the expected detection delay (EDD), also called "mean time to detection (MTD)". Specifically, a change detector processes a stream that contains a change at a known observation κ ($\kappa = 1, \ldots, t$) and we want to know the delay until the change is reported $\mathbb{E}_{x_1,\ldots,x_{\kappa-1}\sim\mathbb{P},x_\kappa,\ldots,x_t\sim\mathbb{Q}}\tilde{T}$, with $\mathbb{P}$ the pre-change distribution and $\mathbb{Q} \neq \mathbb{P}$ the post-change distribution. In other words, the EDD quantifies how many samples of $\mathbb{Q}$ must be processed to flag a change after having observed $\kappa - 1$ samples from $\mathbb{P}$. Note that κ must be chosen large enough to allow MMD to capture the difference in $\mathbb{P}$ and $\mathbb{Q}$, and we assume that the statistic does not exceed the treshold on the first κ samples. In a static setting, the EDD is comparable with the type II error. Note that existing literature (Xie & Siegmund, 2013; Wei & Xie, 2022) usually considers a fixed threshold b for the stopping time for both ARL and EDD, while our threshold ϵ_α depends on the position of the split considered. Experiments for a fixed value of b are deferred to Appendix D.1.

To approximate the ARL and the EDD in different scenarios, we simulate 5-dimensional data distributed according to the multivariate normal $\mathcal{N}(\mathbf{0}, \mathbf{I}_5)$, the uniform $\mathcal{U}[-\mathbf{1}_5, \mathbf{1}_5]$, the Laplace$(0, \sigma\mathbf{I}_5)$, and a mixed distribution, respectively. The mixed distribution is taken to be $\mathcal{N}(\mathbf{0}, \mathbf{I}_5)$ with probability 0.3 and $\mathcal{N}\left(\mathbf{0}, \sigma^2\mathbf{I}_5\right)$ with probability 0.7, where $\mathbf{1}_d$ denotes a vector of d ones and $\mathbf{I}_d$ is the d-dimensional identity matrix. We set $\sigma = 3$.

To compute ARL, we consider 10,000 observations distributed according to either the uniform, the Laplace, or the mixed distribution. Hence, the data does not contain any changes. For MTD, we first run both algorithms on 512 ($= 2^9$) and 1024 ($= 2^{10}$) observations, respectively, leading to MMDEW summarizing the data in one window in both cases. These observations are distributed according to $\mathcal{N}(\mathbf{0}, \mathbf{I}_5)$ and then followed by either the uniform, Laplace, or mixed distribution. That is, we induce a change point at $\kappa = 513$ (resp. $\kappa = 1025$), and then count the number of observations processed from the new distribution until the algorithms report a change.

Figure 3 collects the average results over 20 repetitions. The left plot shows that an increase in the level $\alpha \in (0, 1)$ leads to a decrease in ARL. This is expected as the test becomes more sensitive, leading to more false positives. The baseline achieves a higher ARL but at the cost of an increased runtime. The MTD plots (center and r.h.s.) mirror the ARL observation: The MTD decreases with increasing α. We further observe that the detection delay depends on the post-change distribution. The delay is comparably large when changing from the multivariate standard normal to the mixed distribution. This matches our intuition: the mixed distribution is relatively similar to the pre-change distribution, rendering it difficult to detect a change between them. For larger values of α, that is, $\alpha \geq 0.2$, MMDEW performs similarly to the baseline in all cases. Comparing the MTD when the change happens after 512 observations to the MTD when the change happens after 1024 observations, the results show that more pre-change samples render the algorithms more sensitive to detecting changes, due to more samples improving the approximation of the mean embedding. Overall, the results on these synthetic streams indicate that MMDEW is (i) robust to the choice of α and (ii) that α has the expected influence on the behavior of the algorithm.

[6] For computational reasons, we compute MMD as described in the discussion following Proposition 1. F a fair comparison, we use the distribution-free bound of Proposition 1 for both algorithms.

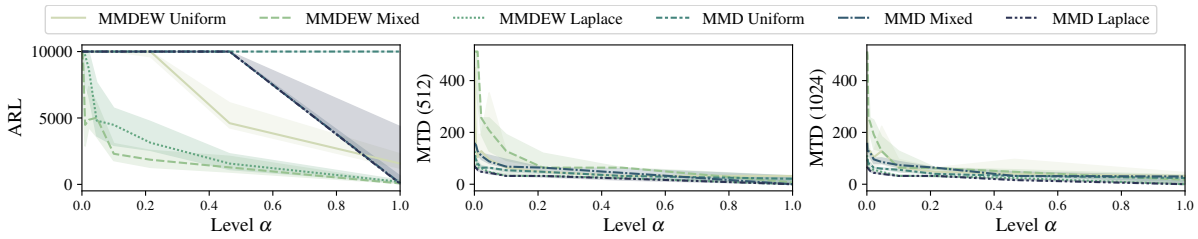

Figure 3: Average run length (ARL) and expected detection delay / mean time to detection (EDD / MTD) of MMDEW on synthetically generated data.

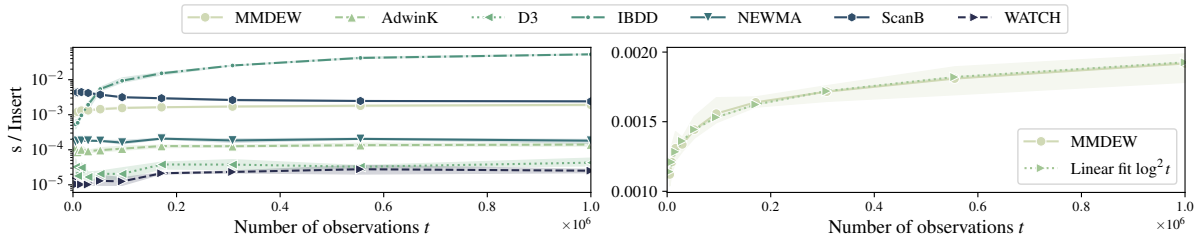

Figure 4: Comparison of runtimes per insert operation (l.h.s.) and least squares fit validating the theoretical runtime complexity of MMDEW w.r.t. the runtime observed in practice (r.h.s.).

Runtime. We now compare the runtime of MMDEW to that of its contenders and additionally validate the runtime guarantees that we derived analytically in Section 4.3.

To this end, we generate a constant stream of 10^6 one-dimensional observations, that is, the observed stream contains no change. Note that, while the dimensionality of the data affects the runtime depending on the used kernel, its influence is the same across all kernel-based algorithms, hence we limit our considerations to the univariate case.

Figure 4 shows the average results over 10 runs. The left plot reveals that the fixed cost per insert of MMDEW is relatively large, as processing a small number of observations requires comparably much time. However, the runtime does not increase by much with the number of observations. The figure also shows that the proposed algorithm's runtime is better than that of an alternate kernel-based method, Scan B-statistics, where we use a window size of $\omega = 100$ in the runtime experiments. For $t > 0.05 \cdot 10^6$, MMDEW also outperforms IBDD. Still, the other algorithms run faster than MMDEW but achieve a lower F_1 score in our later experiments.

The right plot of Figure 4 verifies the analytically derived runtime of $\mathcal{O}\left(\log^2 t\right)$ by fitting the corresponding curve $(t \mapsto c \log^2 t)$ to the measured data with the least squares method. The resulting mean squared error is approximately 10^{-6}, which confirms the preceding asymptotic runtime analysis.

5.2 Real-world classification data

To obtain our change detection quality estimates, we use well-known classification data sets and interpret them as streaming data.[7] This is common in the literature, for example, Faithfull et al. (2019); Faber et al. (2021), as only few high-dimensional annotated change detection data sets are publicly available.

For each data set, we first order the observations by their classes; a change occurs if the class changes. To introduce variation into the order of change points, we randomly permute the order of the classes before each run but use the same permutation across all algorithms. For preprocessing, we apply min-max scaling to all data sets. Table 2 summarizes the data sets, where n is the number of observations, d is the data dimensionality, and #CP is the number of change points.

[7] While MMDEW is not limited to Euclidean data, Euclidean data is the type of data most frequently encountered in practice, and our experiments target at this setting.